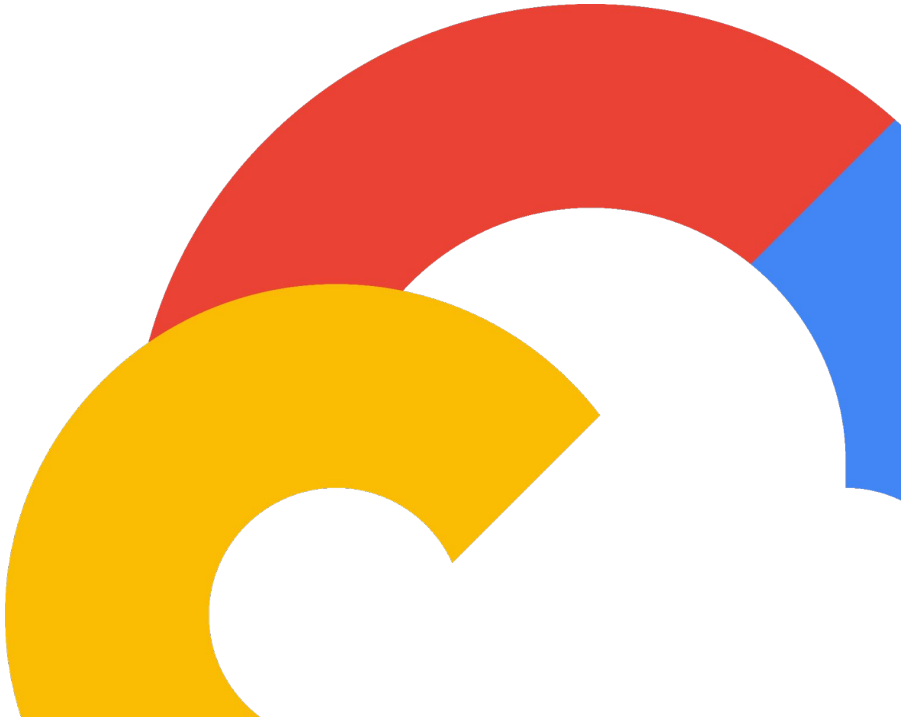
Supercharge Pod Startup with Pod Snapshots

Your Pod's "Skip Intro" Button

November 11, 2025

Google Cloud



Brandon Royal
Senior Product Manager

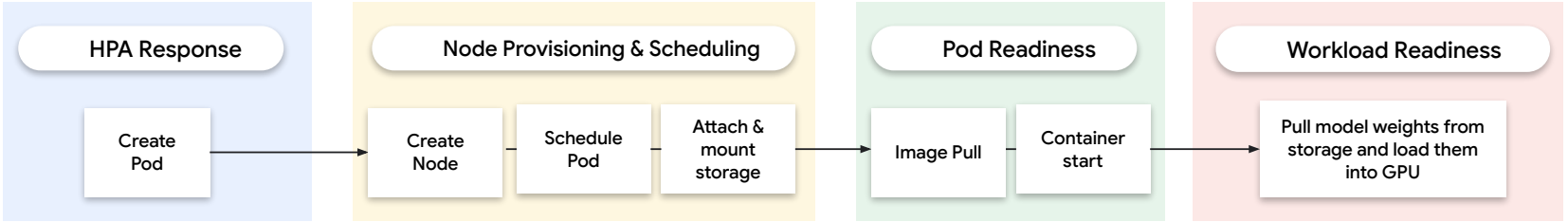
Saket Jajoo
Software Engineer

Slow Workload Startup ⌛

Applications that suffer from slow startup

- AI/ML workloads
- Agentic AI
- Complex Apps
- Gaming Applications
- Interactive Notebooks

Optimization Techniques on GKE



Improved HPA

2x faster scaling with improved metrics and consistent performance

Fast autoscaling for GKE Autopilot

Up to 7x faster pod scheduling time

Improved application response times

3x faster HPA calculations

Container image preloading

Image Streaming

Cloud Storage FUSE

Using HdML Disks

AI Workloads Introduce Scale Challenges

- Large container image size (in the 10's of GBs).
- Large model weights.
- High specialized resource requirements per pod.
- Low response throughput and parallel request processing.

For example, a Llama3-70b workload takes **5.5 minutes** to start up on a g2-standard-96 node with 8 x Nvidia L4 GPUs!

Instant Pod Startup

What if a pod could start ~instantly?

Instead of a cold start, imagine restoring a pod that is already initialized previously. This means starting from a “ready state” where all the preparation work is already done.

- Model weights are already loaded into GPU memory.
- Application caches are fully hydrated and “warm”.
- Dependencies & libraries are initialized in memory.

GKE Pod Snapshots

GKE Pod Snapshots 

A robust, end-to-end managed solution on top of gVisor to make workload checkpointing and restoring a seamless experience in GKE.

CRD-driven: Managed via declarative Kubernetes resources you're already familiar with.

Automated: Handles the entire lifecycle, from capturing the snapshot to storing it in GCS and restoring new pods.

Workload agnostic: Designed for a wide range of stateful applications, from AI/ML to complex monoliths.

Up to

80%

Start Latency Improvement for Large (70B parameter) Workloads

Benchmark Results

GKE Pod Snapshots were integrated with Vertex AI's Model Garden service to accelerate the pod startup performance for the top 3 most popular models.

Model	Time take to start the workload from scratch	Time take to restore the workload from a snapshot	Performance improvement
Llama3-8b (small)	83.3s	15.4s	**~81.5**
Gemma2-27b (medium)	151.8s	35s	**~77%**
Llama3.3-70b (large)	330.5s	79.8s	**~76%**

These benchmarks have been performed on g2-standard GKE nodes with Nvidia L4 GPUs.

gVisor 

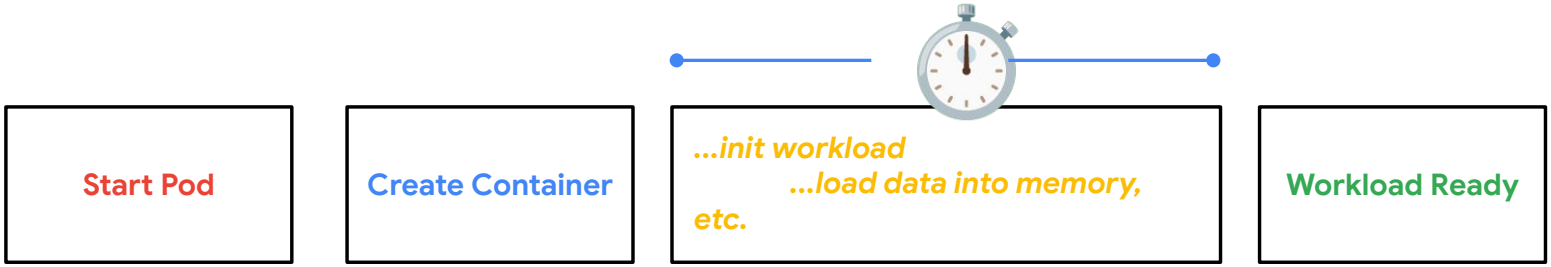
gVisor is a sandboxed container runtime (used in GKE) providing application kernels for containers.

- Checkpoint/restore is a first class feature.
- Understands the construct of a “pod” (a.k.a. a sandbox in gVisor) and hence can perform multi-container checkpoint/restore.
- Provides enhanced checkpoint/restore performance (async I/O, multi-file checkpoint format, Direct I/O).
- Provides security out-of-the-box as gVisor sandbox limits access to host kernel.

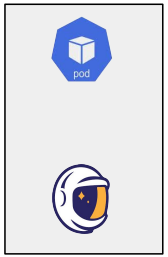
What gets checkpointed / restored

- **CPU & Memory State**: The exact in-memory state of all processes using gVisor
- **GPU Memory**: Using NVIDIA's `cuda-checkpoint` binary, we save the entire state of the GPU memory.
- **Filesystem**: The container's rootfs changes, tmpfs mounts and any `emptyDir` volumes are saved.
- **Communication**: Sockets, devices, pipes, loopback connections, listening sockets.

Normal Pod Creation Workflow



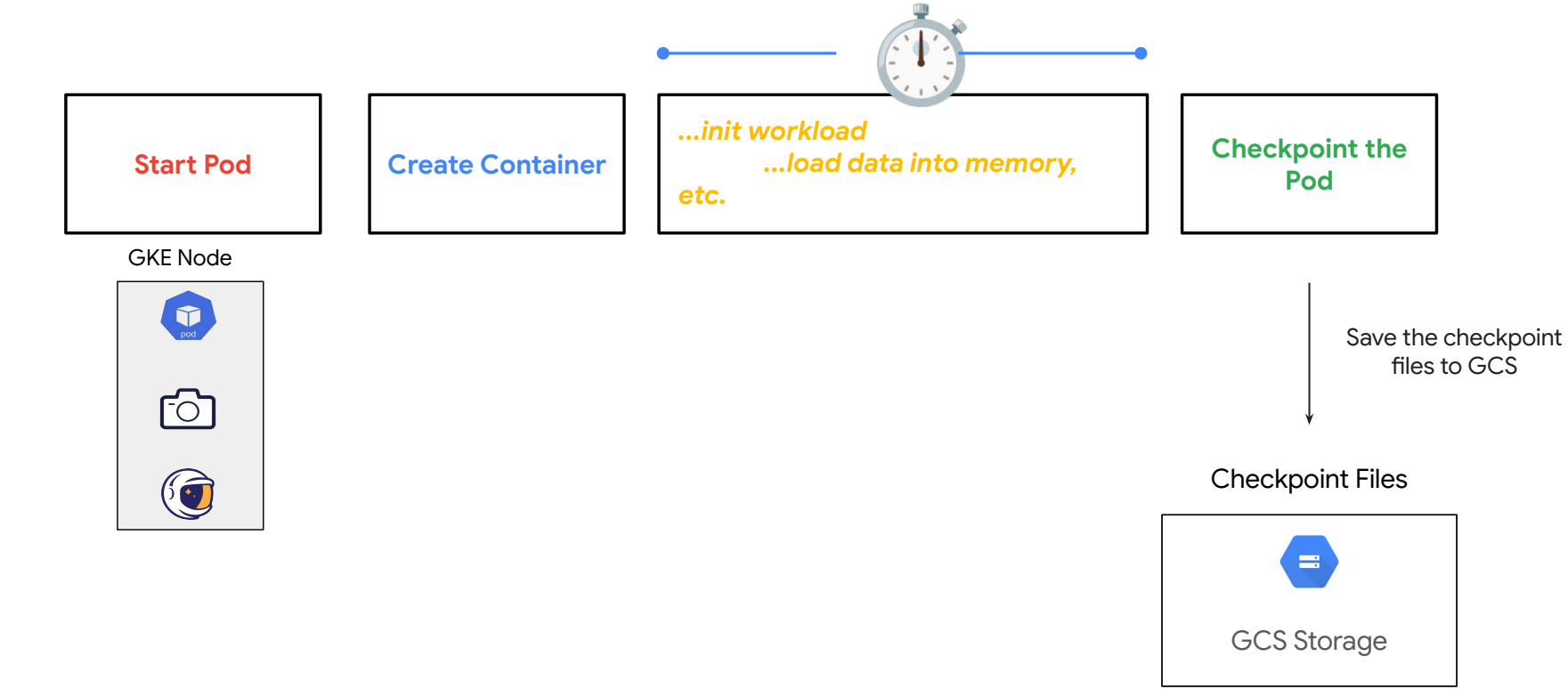
GKE Node



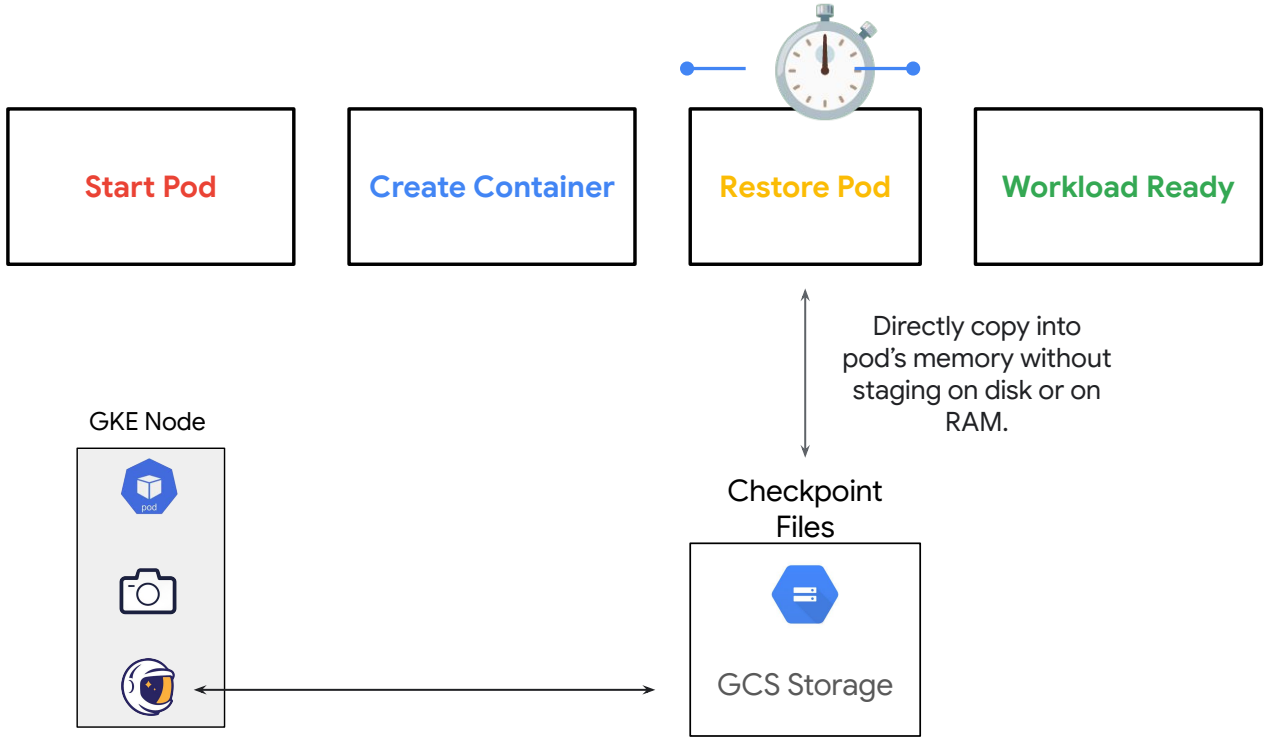
For example,

- Initializing large application dependencies and libraries
- Reading model weights into GPU memory

GKE Pod Snapshot Checkpoint Workflow



GKE Pod Snapshot Restore Workflow



User Experience - PodSnapshotStorageConfig

Define a storage configuration

First, tell GKE where to store the snapshots by creating a `PodSnapshotStorageConfig` CR.

For Public Preview, we support Google Cloud Storage. Just provide a bucket name and an optional path. GKE will handle the rest.

```yaml
apiVersion: podsnapshot.gke.io/v1alpha1
kind: PodSnapshotStorageConfig
metadata:
  name: pod-snapshots-storage
spec:
  snapshotStorageConfig:
    gcs:
      bucket: "pod-snapshots"
      path: "my-cool-ML-model"
```

User Experience - PodSnapshotPolicy

Enable with a Simple Policy

Simply define a `PodSnapshotPolicy` that selects your workload and specifies the behavior without the overhead of managing individual snapshots.

The system handles the rest: it automatically creates new snapshots when the workload changes and restores them for new replicas.

```yaml
apiVersion: podsnapshot.gke.io/v1alpha1
kind: PodSnapshotPolicy
metadata:
  name: snapshot-my-cool-ML-pod
  namespace: default
spec:
  storageConfigName: pod-snapshots-storage
  selector:
    matchLabels:
      app: ML-training
  triggerConfig:
    type: workload
```

User Experience - PodSnapshotPolicy Triggers

Workload-based Trigger

A workload-based trigger is useful when the workload can determine the exact moment that it is ready to trigger a checkpoint (e.g. a workload can trigger a checkpoint once it finishes loading the model weights in memory and is ready to receive inference requests).

Other trigger types will include

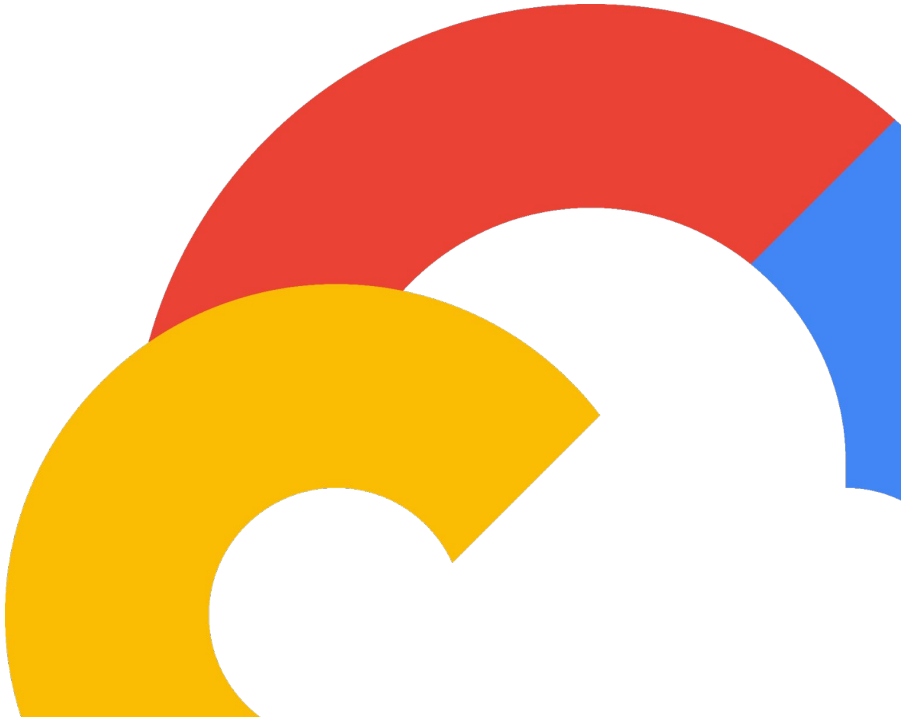
- Manual Trigger (experimental).
- Pod's Readiness probe.

```
def ready_for_snapshot():
  """
  Triggers a checkpoint and waits until it's safe to
  resume.
  """
  try:
    with open("/proc/gvisor/checkpoint", "r+") as f:
      f.write("1")
      res = f.read()
      print(f"Snapshot complete: {res}")
  except OSError as e:
    return e
```

Considerations

- **Hardware Consistency:** Snapshots must be restored to the same CPU/GPU architecture, GPU driver version, machine type, gVisor version.
- **Pod Spec Binding:** Snapshots are tied to a specific pod spec. Changes to the image, args, volumes, etc. will trigger a new snapshot.
- **Network:** Non-loopback and non-listening type network connections must be re-established upon restore.
- **Application State:** State that needs to be unique per-pod (like encryption keys, session IDs, or kernel-provided random sources like `/dev/random`) must be re-initialized or re-seeded by the application post-restore.
- **Environment Variables**: A special file (`/proc/gvisor/spec_environ`) is created upon restore, which contains updated environment variables defined in the Pod spec of the restored Pod, that the application can read form and update their in-memory environment variables.

Thank you



Google Cloud